SMARTLOGIX

TEACHER PRESENTATION & VIVA DEFENSE GUIDE

A minimal, structured guide with direct window screenshots to easily identify each UI screen, live demonstration scripts, DAA algorithmic mechanics, and anticipated evaluator defense questions.

> 📋 **STUDENT PRESENTATION PROTOCOL**
>
> *This guide is your complete, battle-ready manual for presenting the **SmartLogix** project to your evaluators and professors. It is written directly against the actual codebase, live routes, algorithms, and PostgreSQL schema. Keep the application open in your browser (`http://localhost:5173`) while presenting.*
>
> ---

NAVIGATION MATRIX / TABLE OF CONTENTS

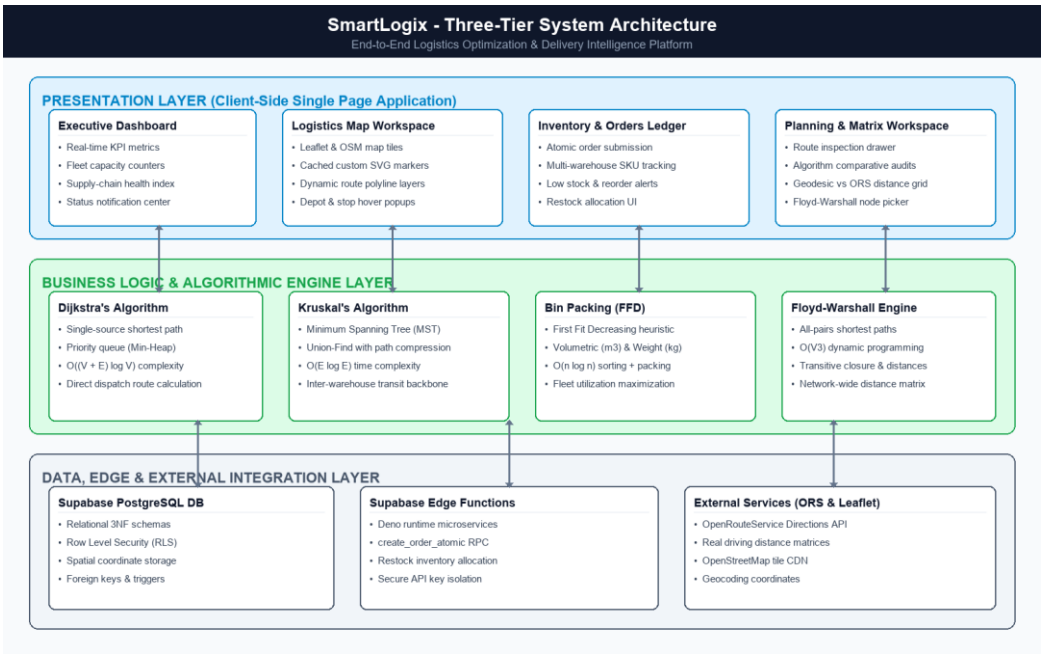
Part I: Live Demonstration & Workflow	Part II: Algorithms & Viva Defense Playbook
1. Project in One Minute (Elevator Pitch)	10. How to Show Each Algorithm Live (Checklist)
2. The Big Picture Architecture & Tech Stack	11. 'Why This Algorithm?' — Defense FAQ (13 Q&As)
3. Complete 12-Step Demonstration Workflow	12. 'Show Me the Code' Questions & Function Map
4. Algorithm → Real Feature Mapping Table	13. Algorithm Complexity Cheat Sheet Matrix
5. Deep Dive on Each of the 6 Algorithms	14. Presentation Timing Scripts (5, 10, 15 Min)
6. 'Where is DAA Actually Used?' (Defense)	15. Likely Viva Questions & Technical Answers
7. Database Architecture & Algorithm Sourcing	16. Trick Questions & How to Defend Them
8. Complete Data Flow Diagrams (Fulfillment/Plan)	17. Important: What NOT to Claim
9. Live Demo — Exact Word-for-Word Script	18. Final One-Page Quick Reference Cheat Sheet

1. Project in One Minute

> **ELEVATOR PITCH SCRIPT (VERBATIM SPOKEN DIALOGUE)**
>
> *"Good morning, Professor. Today I am presenting **SmartLogix**, an intelligent inventory management and delivery logistics optimization system.*
>
> *Traditional supply chain systems are merely reactive CRUD applications—they store static inventory records and log customer addresses, leaving routing decisions, warehouse selection, and restock distribution to human guesswork. SmartLogix solves this by embedding classical **Design and Analysis of Algorithms (DAA)** directly into real-world logistics workflows.*
>
> *Instead of relying on unverified heuristics or straight-line Euclidean distance, SmartLogix models the logistics network as a weighted road graph. When an order arrives, it runs **Dijkstra's Algorithm** over actual road kilometers to recommend the optimal fulfillment warehouse. When incoming bulk stock arrives, it applies **First-Fit Decreasing (FFD) Bin Packing** across variable warehouse storage capacities to prevent overflow. For fleet dispatches, it solves the **Traveling Salesperson Problem (TSP)** using an exact **Branch & Bound** solver with admissible lower bound pruning alongside a fast **Greedy Nearest Neighbor** heuristic. Finally, for network infrastructure planning, it computes **Floyd-Warshall** all-pairs shortest paths and **Kruskal's Minimum Spanning Tree** using a **Disjoint Set (Union-Find)** with path compression and rank union.*
>
> *Every algorithm runs on actual database records and visualizes live on our interactive Leaflet road map. Let me walk you through the system."*
>
> ---

2. The Big Picture Architecture

System Flow & Component Hierarchy



WINDOW PREVIEW: System Architecture Flow: Client UI, Pure TypeScript Engine & Database



Technology Breakdown & Purpose

Technology	Where Used & Purpose
React 18	Single Page Application framework with code-splitting (**`React.lazy`**) for performance.
TypeScript	Strict compile-time typing for database schemas, algorithm inputs, graph edges, and API payloads.
Tailwind CSS	Custom styling, responsive flex/grid layouts, badges, and modal dialogs.
Supabase (PostgreSQL)	Relational persistence, relational integrity, row-level storage, and 9 transactional atomic RPCs.
Leaflet & React-Leaflet	Interactive geospatial map engine rendering warehouse markers, delivery pins, TSP tours, Dijkstra paths, and MST edges.
OpenRouteService (ORS)	Invoked via Supabase Edge Function (**`ors-matrix`**) to retrieve real-world driving distances and durations between coordinates.
Recharts	Telemetry charts in Reports (Order status distribution, vehicle fleet capacity utilization).
Framer Motion	Micro-interactions, animated metric cards, and smooth modal appearances.

3. Complete Demonstration Workflow

Follow this step-by-step sequence during your presentation. Screenshots are placed at each step so you can instantly recognize the active window.

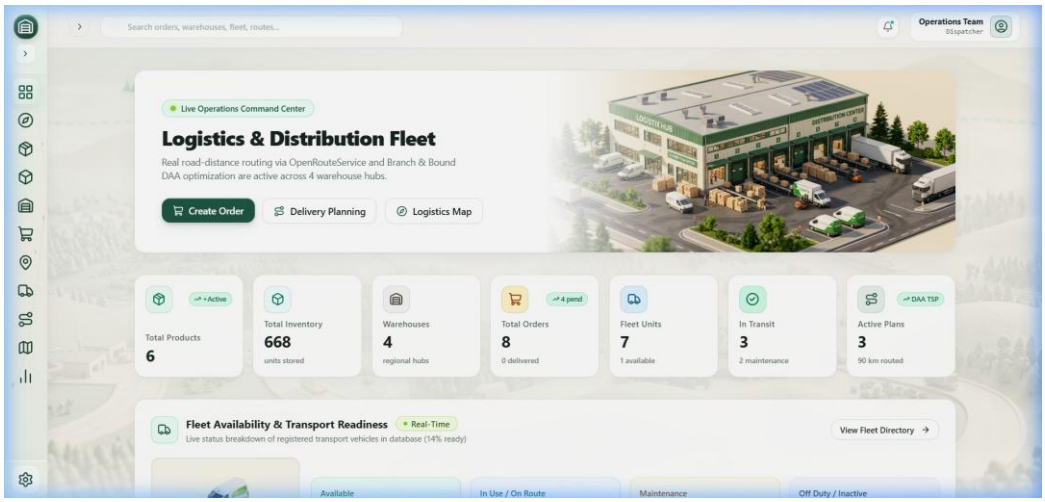
Step 1: Dashboard & System Health

ALGORITHM: Explicitly state: **No DAA algorithm here** (Operational aggregation). | **DATABASE:** Tables: `warehouses`, `orders`, `vehicles`, `delivery_plans`.

> 💻 **WHAT I DO (ACTION)**
>
> Open **`http://localhost:5173/`** (Dashboard).

> 🎙️ **WHAT I SAY (SPOKEN SCRIPT)**
>
> *"We start at the Operations Dashboard. This displays live aggregate metrics directly queried from PostgreSQL. Notice our fleet readiness and warehouse storage capacity. Everything here feeds into our downstream optimization engines."*

> 👁️ **WHAT THE TEACHER SEES (SCREEN DISPLAY)**
>
> Metric cards showing Active Warehouses, Fleet Vehicles, Pending Orders, System Capacity Utilization, Quick Actions, and Recent Dispatches.



WINDOW PREVIEW: Dashboard & System Health Overview `[Route: /]`

> ⚙️ **TECHNICAL & DAA CONNECTION**
>
> **`Dashboard.tsx`** loads data via **`Promise.all`** across **`api.warehouses.list()`**, **`api.orders.list()`**, **`api.vehicles.list()`**, and **`api.plans.list()`**.

Step 2: Warehouses & Capacity Foundation

ALGORITHM: Foundation for **Bin Packing** and **TSP Depot**. | **DATABASE:** Table: `warehouses`.

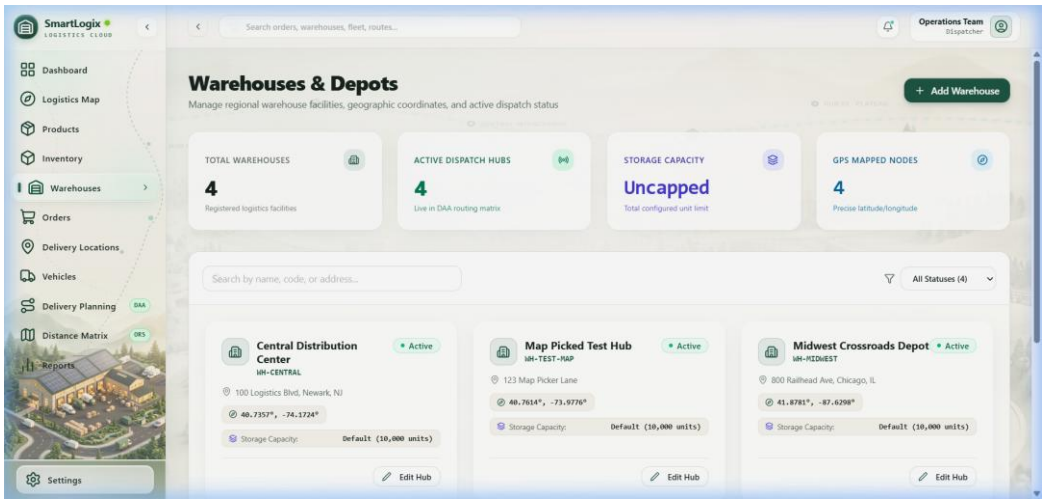
> 💻 **WHAT I DO (ACTION)**
>
> Click **Warehouses** in the sidebar (**`/warehouses`**).

🎙️ WHAT I SAY (SPOKEN SCRIPT)

"Here are the physical depots. Each warehouse has an exact GPS coordinate and a finite `storage_capacity`. Warehouses serve two critical algorithmic roles: they are the depots (source vertex 0) for delivery routes, and they act as bounded bins during restocking."

👁️ WHAT THE TEACHER SEES (SCREEN DISPLAY)

List of fulfillment centers (e.g., Central Distribution Hub, North Sub-Hub) with assigned physical coordinates (Latitude/Longitude), contact info, and configured **`storage_capacity`**.



WINDOW PREVIEW: Warehouses & Storage Capacity Table `[Route: /warehouses]`

⚙️ TECHNICAL & DAA CONNECTION

`storage_capacity` is a dedicated column on **`warehouses`**, used to compute available capacity for Bin Packing.

Step 3: Products & Inventory Tracking

ALGORITHM: Preparation for **Bin Packing**. | **DATABASE:** Tables: `products`, `inventory`, `warehouses`.

💻 WHAT I DO (ACTION)

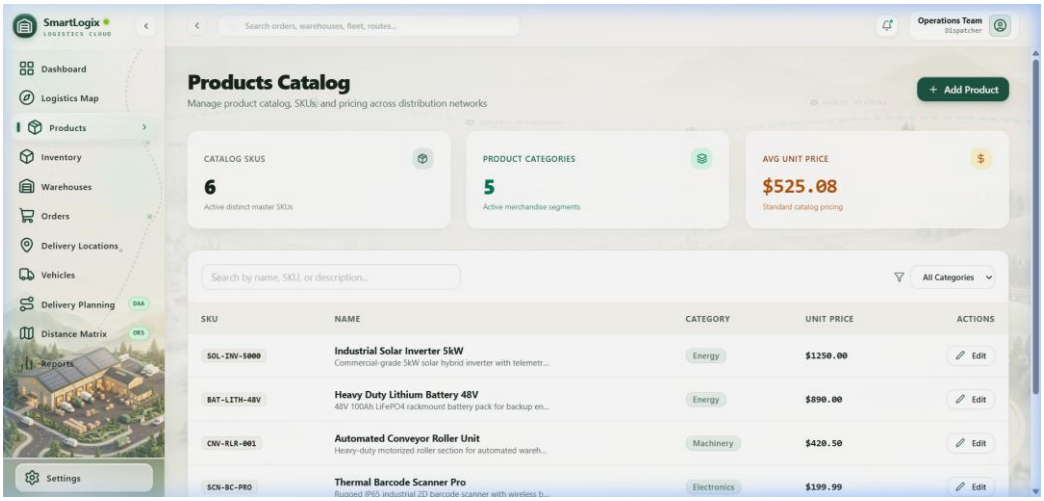
Click **Products** (**`/products`**) and then **Inventory** (**`/inventory`**).

🎙️ WHAT I SAY (SPOKEN SCRIPT)

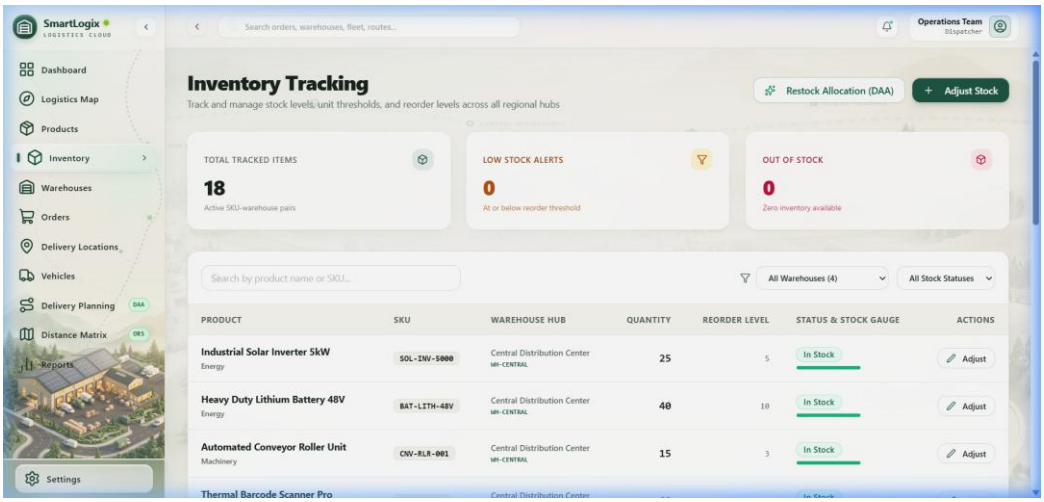
"Our inventory table joins products with warehouses. We track real-time stock levels against reorder thresholds. When inventory runs low, rather than arbitrarily ordering stock to a single warehouse, SmartLogix optimizes replenishment using Bin Packing."

👁️ WHAT THE TEACHER SEES (SCREEN DISPLAY)

Products with SKU, weight, and volume. The Inventory page displays current stock counts per warehouse with stock level badges (Normal, Low Stock, Out of Stock).



WINDOW PREVIEW: **Products Catalog & SKU Specifications** `[Route: /products]`



WINDOW PREVIEW: **Warehouse Inventory Tracking & Reorder Levels** `[Route: /inventory]`

> ⚙️ **TECHNICAL & DAA CONNECTION**
>
> **`InventoryPage.tsx`** loads inventory records joined with **`products`** and **`warehouses`**.

Step 4: First-Fit Decreasing (FFD) Bin Packing Live Demo

ALGORITHM: **First-Fit Decreasing Bin Packing** ($O(m \log m)$). | **DATABASE:** Queries `warehouses` and `inventory`; updates `inventory` via `addStock`.

> 💻 **WHAT I DO (ACTION)**
>
> On **`/inventory`**, click the top-right button: **"Restock Optimization (Bin Packing)"**.

> 🎙️ **WHAT I SAY (SPOKEN SCRIPT)**
>
> *"This is our first core DAA algorithm: First-Fit Decreasing Bin Packing (`src/algorithms/binPacking.ts`). Each warehouse acts as a variable-capacity bin where available room is `storage_capacity - occupied_capacity`. The algorithm sorts warehouses descending by available capacity and assigns the incoming stock lot to the largest available facilities first. This minimizes storage fragmentation and prevents depot overflow. Clicking 'Apply Allocation' atomically persists this stock across warehouses."*

👁️ WHAT THE TEACHER SEES (SCREEN DISPLAY)

A detailed algorithmic breakdown table appears showing:

- Warehouses evaluated and utilized.
- Available capacity per warehouse before allocation.
- Allocated quantity assigned to each warehouse.
- Remaining capacity after allocation.
- Execution time in milliseconds (e.g., `< 0.5 ms`).

⚙️ TECHNICAL & DAA CONNECTION

Calls **`solveRestockBinPacking()`** synchronously on the client, then calls **`api.inventory.applyRestockAllocation()`** to update PostgreSQL inventory records.

Step 5: Delivery Locations & Road Distance Graph

ALGORITHM: Weighted Adjacency Matrix formulation ($G = (V, E, W)$). | **DATABASE:** Table: `location_distances`.

💻 WHAT I DO (ACTION)

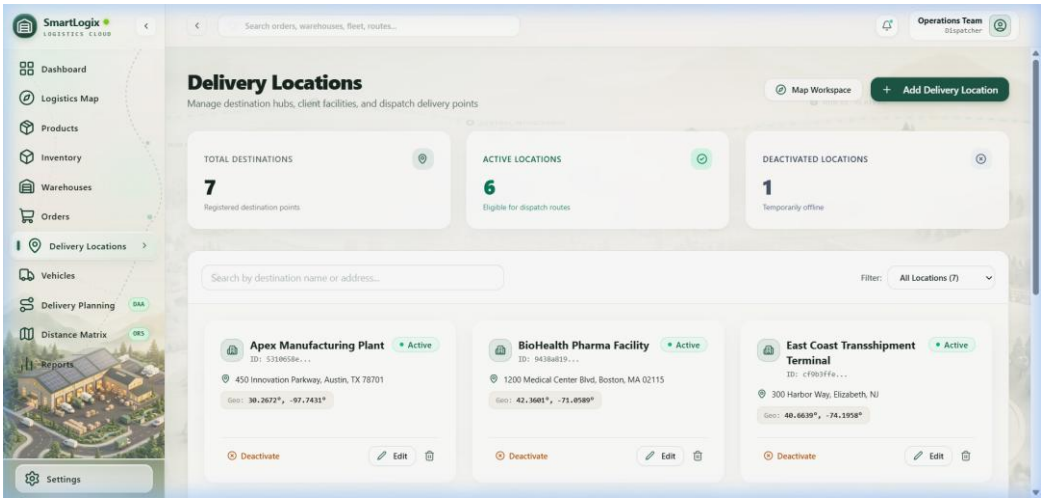
Click **Delivery Locations** (**`/locations`**), then **Distance Matrix** (**`/distance-matrix`**).

🎙️ WHAT I SAY (SPOKEN SCRIPT)

"SmartLogix does not make the mistake of using Euclidean straight-line distance. In logistics, rivers, one-way streets, and highways dictate real travel. Our distance matrix stores verified road network distances in the `location_distances` table, sourced from OpenRouteService. This distance matrix forms the weighted adjacency matrix for our shortest path and routing algorithms."

👁️ WHAT THE TEACHER SEES (SCREEN DISPLAY)

The active customer drop-off locations with GPS coordinates. On **`/distance-matrix`**, a grid shows pairwise road distances in kilometers.



WINDOW PREVIEW: **Delivery Locations & Road Graph Vertices** `[Route: /locations]`

> ⚙️ **TECHNICAL & DAA CONNECTION**
>
> Stored in **`location_distances`** with origin, destination, **`distance`** (km), **`distance_meters`**, and **`duration_seconds`**.

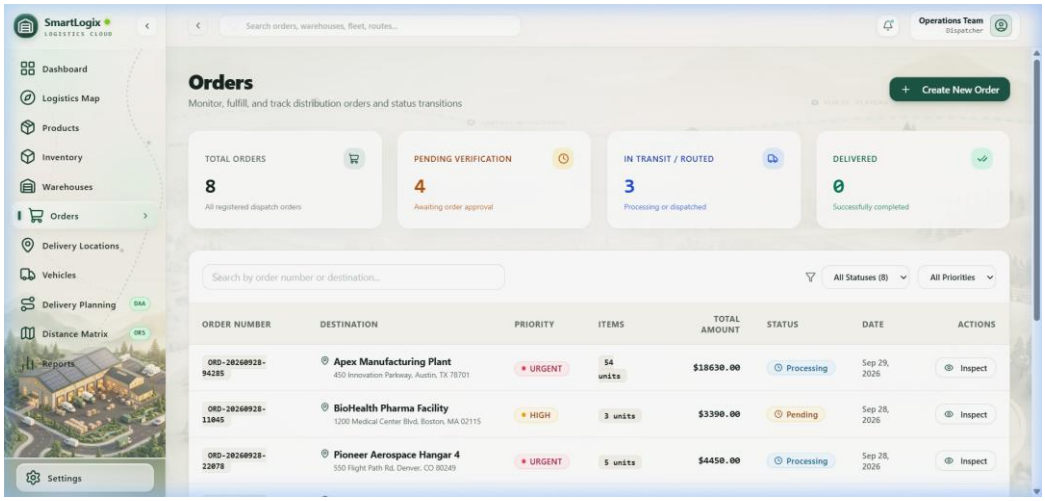
Step 6: Dijkstra Warehouse Recommendation (Create Order)

ALGORITHM: **Dijkstra's Shortest Path Algorithm** ($O((V+E)\log V)$). | **DATABASE:** Queries `location_distances`, `inventory`, `warehouses`; executes RPC `create_order_atomic`.

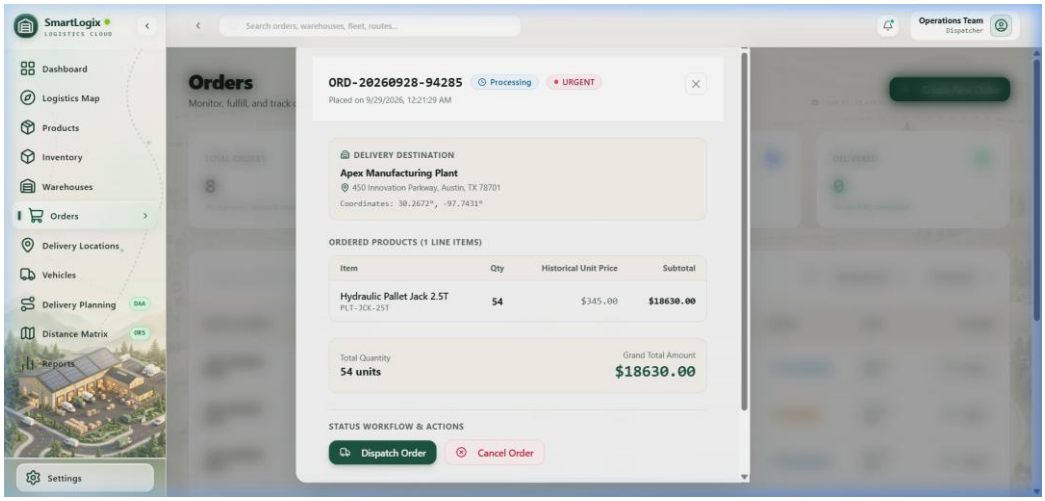
> 💻 **WHAT I DO (ACTION)**
>
> Click **Orders** (**`/orders`**), then click **"+ Create Order"** (**`/orders/create`**).

> 🎙️ **WHAT I SAY (SPOKEN SCRIPT)**
>
> *"When a customer places an order, which warehouse should fulfill it? SmartLogix runs Dijkstra's Algorithm (`src/algorithms/dijkstra.ts`). It constructs a graph of road distances between all warehouses and the target destination. It filters warehouses that actually possess sufficient inventory and selects the one with minimum road distance. Notice how clicking 'Accept Recommendation' automatically selects that warehouse in the order."*

> 👁️ **WHAT THE TEACHER SEES (SCREEN DISPLAY)**
>
> A candidate ranking table showing:
>
> - Each warehouse's road distance to the customer (e.g., **`12.4 km`**).
> - Stock availability check (Has Sufficient Stock: YES/NO).
> - Recommended status (Best Candidate highlighted with shortest road travel).
> - Execution time (**`0.12 ms`**).



WINDOW PREVIEW: **Orders Management & Dijkstra Recommended Depots** `[Route: /orders]`



WINDOW PREVIEW: **Order Line Items & Stock Reservation Modal** `[Route: /orders/create]`

> ⚙️ **TECHNICAL & DAA CONNECTION**
>
> Executes **`solveDijkstra()`** against the graph constructed from **`location_distances`**. Submitting the order calls **`create_order_atomic`** RPC in PostgreSQL.

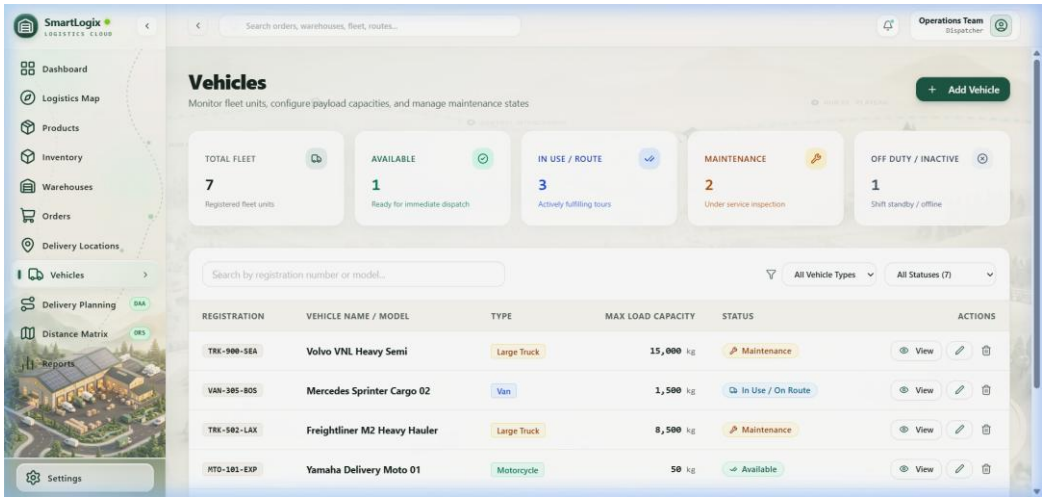
Step 7: Delivery Planning & Vehicle Capacity Validation

ALGORITHM: Vehicle capacity constraint checking. | **DATABASE:** Tables: `delivery_plans`, `delivery_plan_orders`, `vehicles`.

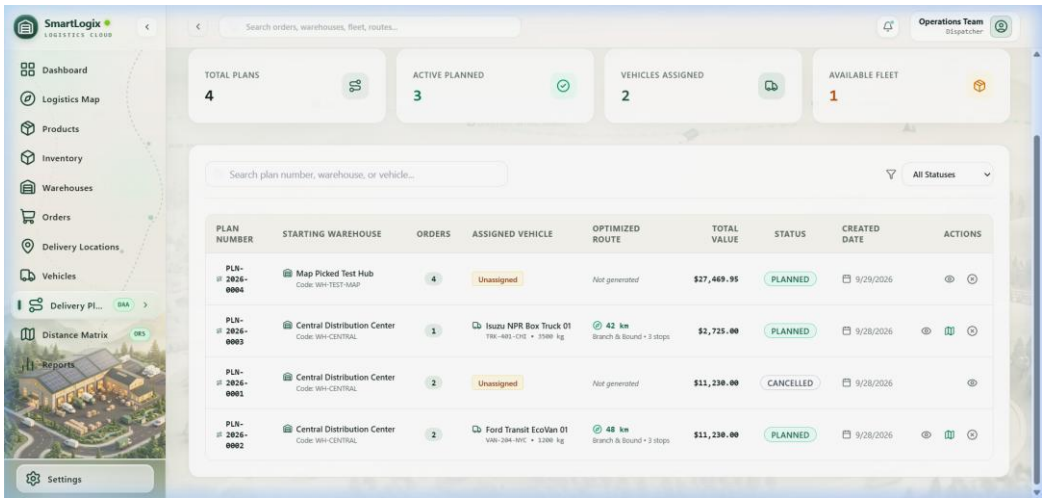
> 💻 **WHAT I DO (ACTION)**
>
> Click **"+ Create Delivery Plan"**, select the warehouse, check 3 or 4 eligible orders, and click **"Create Plan"**.

> 🎙️ **WHAT I SAY (SPOKEN SCRIPT)**
>
> *"Once orders are created, dispatch managers group them into delivery plans. Here you can see pending orders awaiting assignment to a warehouse run."*

> 👁️ **WHAT THE TEACHER SEES (SCREEN DISPLAY)**
>
> Vehicle selector displays vehicles with their max payload capacity and status (**`AVAILABLE`**). If an order load exceeds vehicle capacity or if units mismatch (e.g., **`m3`** vs **`kg`**), the system enforces safety validation.



WINDOW PREVIEW: **Fleet Vehicles & Payload Capacity Limits** `[Route: /vehicles]`



WINDOW PREVIEW: **Delivery Dispatch Planning & Vehicle Assignment** `[Route: /planning]`

> ⚙️ **TECHNICAL & DAA CONNECTION**
>
> Calls **`create_delivery_plan_atomic`** RPC, then **`assign_vehicle_to_delivery_plan`** RPC.

Step 8: TSP Route Optimization (Branch & Bound vs Greedy)

ALGORITHM: **Branch & Bound TSP** (Exact with pruning) & **Greedy Nearest Neighbor TSP** ($O(n^2)$). | **DATABASE:** RPC: `save_delivery_plan_route`.

> 💻 **WHAT I DO (ACTION)**
>
> Click **"Save Optimized Route to Plan"**.

WHAT I SAY (SPOKEN SCRIPT)

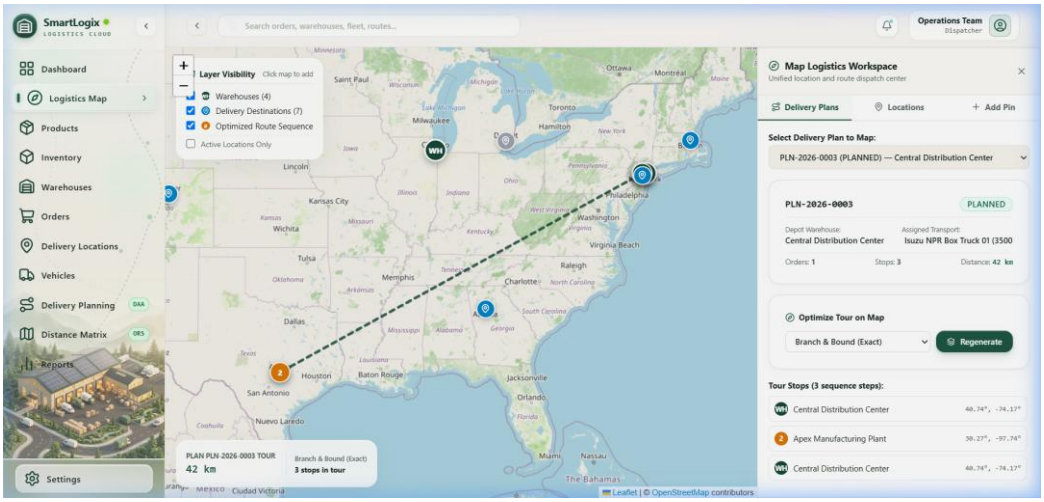
"This is our flagship algorithm demonstration: Traveling Salesperson Problem Optimization (`src/algorithms/tsp.ts`). The dispatch vehicle must depart the warehouse, deliver to all customer locations, and return to the depot with minimum total kilometers.

- *With Branch & Bound, we find the mathematically guaranteed optimal tour. We compute an admissible lower bound at each state and prune branches that exceed our best-known upper bound (initially seeded by Greedy NN). Notice the 'Nodes Pruned' counter—that proves the bounding function is actively avoiding exponential exploration.*
- *When we switch to Greedy Nearest Neighbor, the algorithm greedily hops to the closest unvisited stop in O(n^2) time. It executes in fractions of a millisecond, but for complex graphs, Branch & Bound achieves a shorter total distance."*

WHAT THE TEACHER SEES (SCREEN DISPLAY)

A modal opens showing the selected warehouse and all delivery stops. Notice the algorithm selection toggle:

- **Branch & Bound (Exact Optimal)**
- **Greedy Nearest Neighbor (Heuristic)**



WINDOW PREVIEW: **Map Workspace with TSP Optimized Multi-Stop Tour** `[Route: /map]`

TECHNICAL & DAA CONNECTION

Saves route stops and distance into **`delivery_plans`** via **`save_delivery_plan_route`** RPC.

Step 9: Unified Map Intelligence Layer

ALGORITHM: Spatial projection of **TSP, Dijkstra, Kruskal, Floyd-Warshall**. | **DATABASE:** Reads live `warehouses`, `delivery_locations`, `location_distances`.

WHAT I DO (ACTION)

1. Click **`[TSP Route]`**: Select your active plan. The full dispatch loop renders in royal blue with numbered stop badges (**`WH`**, **`1`**, **`2`**, **`3`**, **`WH`**).
2. Click **`[Dijkstra Shortest Path]`**: Select Origin (e.g., *Central Distribution Hub*) and Destination (e.g., *Metro Retail Outlet*). Click **"Calculate Path"**. The map draws the exact shortest road route in emerald green with distance and hop sequence.
3. Click **`[Kruskal MST]`**: The map displays the Minimum Spanning Tree backbone in dashed purple lines connecting all facilities with minimum total road length.

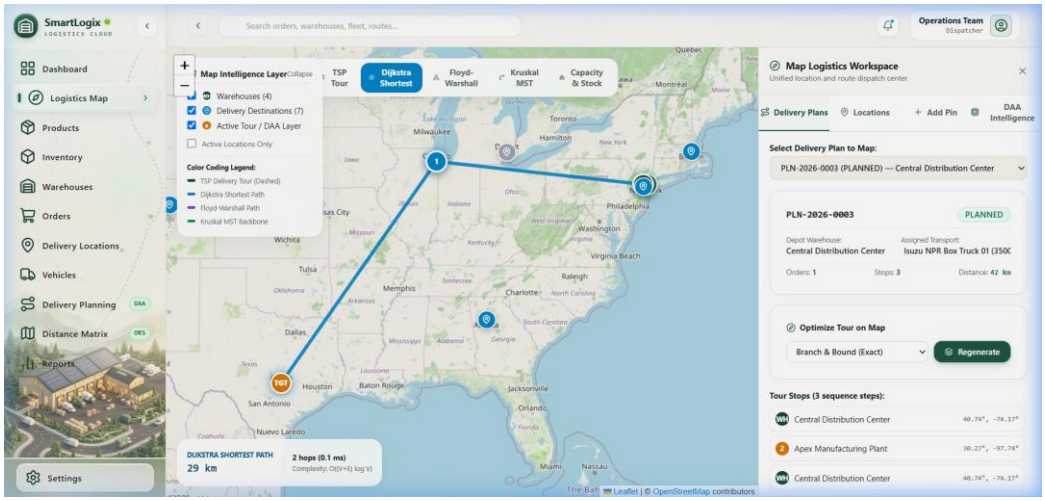
WHAT I SAY (SPOKEN SCRIPT)

"The Map Workspace (`src/pages/MapWorkspace.tsx`) is our unified spatial intelligence layer. Rather than having separate maps for separate features, all five DAA algorithms project onto the exact same real-world geography. We can visually inspect how a closed TSP tour differs from an open Dijkstra shortest path and an acyclic Kruskal spanning tree."

WHAT THE TEACHER SEES (SCREEN DISPLAY)

Fullscreen interactive Leaflet map with custom warehouse depot pins (emerald green), delivery location pins (sky blue), and an Intelligence Mode toolbar at the top:

- `[TSP Route]`
- `[Dijkstra Shortest Path]`
- `[Floyd-Warshall]`
- `[Kruskal MST]`
- `[Bin Packing]`



WINDOW PREVIEW: **Interactive Leaflet Map with Dijkstra Shortest Path Route** `[Route: /map]`

TECHNICAL & DAA CONNECTION

Leaflet **`<Polyline>`** and custom **`<Marker>`** DivIcons rendered dynamically based on algorithm coordinate sequences.

Step 10: Floyd-Warshall All-Pairs & Kruskal MST Deep Dive

ALGORITHM: **Floyd-Warshall** ($O(V^3)$), **Kruskal's MST** ($O(E \log E)$), **Union-Find** ($O(\alpha(V))$). | **DATABASE:** Reads from `location_distances`.

WHAT I DO (ACTION)

Return to **Distance Matrix** (**`/distance-matrix`**) and scroll down to the **Advanced Graph Analysis** tabs.

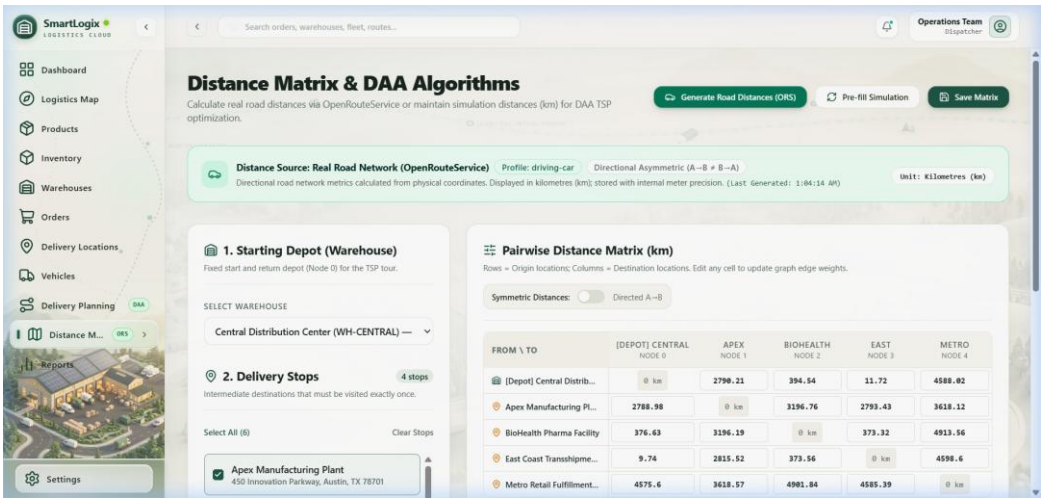
WHAT I SAY (SPOKEN SCRIPT)

*"On this page we expose the graph-theoretic engines:

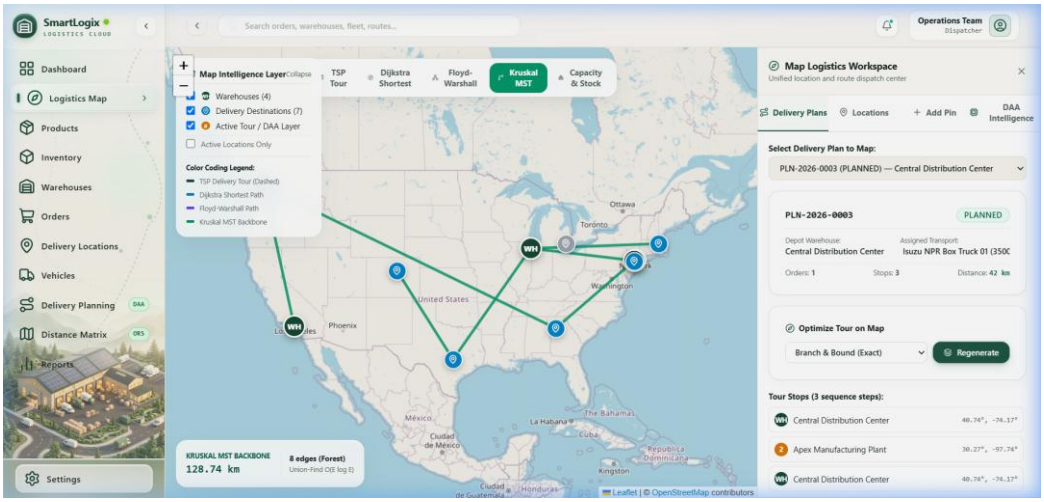
- ***Floyd-Warshall*** (`src/algorithms/floydWarshall.ts`) *computes all-pairs shortest paths using dynamic programming in O(V^3). It solves the problem of finding shortest paths between every warehouse and customer simultaneously, with complete path reconstruction via its predecessor matrix.*
- ***Kruskal's Algorithm*** (`src/algorithms/kruskal.ts`) *with* ***Union-Find*** (`src/algorithms/unionFind.ts`) *computes the Minimum Spanning Tree of our entire logistics network. It selects edges in increasing order of road distance, using path compression and union by rank to detect and discard cycle-forming edges in O(E log E) time."**

WHAT THE TEACHER SEES (SCREEN DISPLAY)

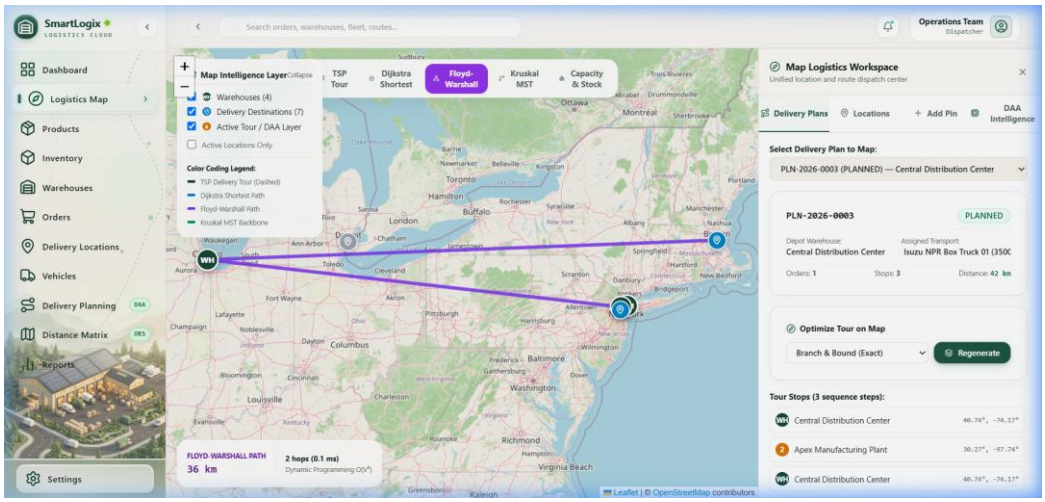
- **Floyd-Warshall Tab**: Select Origin and Destination from dropdowns. The interactive inspector reconstructs the multi-hop shortest path and displays the full dynamic programming distance matrix (D^{(k)}).
- **Kruskal MST Tab**: Displays total network vertices, candidate edges, MST selected edges (|V| - 1), rejected cycle edges, total network backbone cost, and the embedded `MstNetworkMap`.



WINDOW PREVIEW: **Road Distance Matrix Adjacency Graph Table** `[Route: /distance-matrix]`



WINDOW PREVIEW: **Kruskal Minimum Spanning Tree Network Backbone** `[Route: /distance-matrix]`



WINDOW PREVIEW: **Floyd-Warshall All-Pairs Distance Matrix DP Grid** `[Route: /distance-matrix]`

Step 11: Reports & DAA Telemetry Ledger

ALGORITHM: Telemetry analysis of **all algorithms**.

--- | **DATABASE:** None

💻 WHAT I DO (ACTION)

Click **Reports** in the sidebar (**`/reports`**).

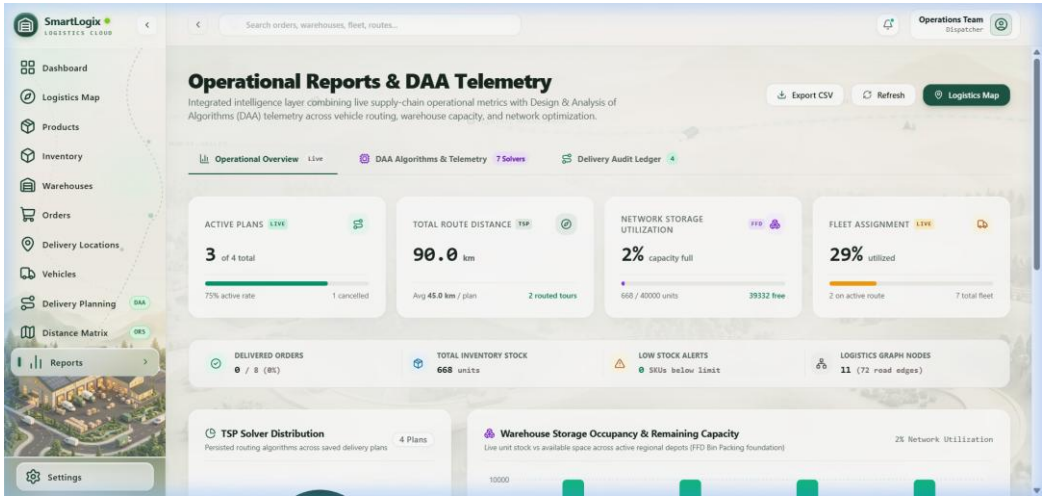
🎙️ WHAT I SAY (SPOKEN SCRIPT)

"Finally, the Reports page (`src/pages/Reports.tsx`) provides formal algorithmic telemetry. Notice how we strictly separate our metrics into four scopes: Live Database state, Persisted Plans, Graph Topology, and DAA Formal Specifications. Every algorithm run records its execution duration in milliseconds, proving sub-millisecond to low-millisecond performance on real operational inputs."

👁️ WHAT THE TEACHER SEES (SCREEN DISPLAY)

Four tabbed analytics views:

- **Overview**: Order status charts, vehicle capacity utilization bar chart.
- **DAA Telemetry**: Algorithm catalog cards, mathematical paradigms, time/space complexities, and live execution runtime benchmarks.
- **Execution Ledger**: Historical log of persisted routes, algorithms executed, total kilometers, and timestamps.



WINDOW PREVIEW: **DAA Telemetry Ledger & Performance Benchmarks** `[Route: /reports]`

> ⚙️ **TECHNICAL & DAA CONNECTION**
>
> Recharts visualization of real PostgreSQL data; catalog mapping of all 6 algorithms.

Step 12: Settings & Algorithm Governance

ALGORITHM: None | **DATABASE:** None

> 💻 **WHAT I DO (ACTION)**
>
> Click **Settings** in the sidebar (**`/settings`**).

> 🎙️ **WHAT I SAY (SPOKEN SCRIPT)**
>
> *"To protect browser responsiveness against the NP-hard nature of TSP, the Settings page allows administrators to govern algorithmic constraints. We restrict exact Branch & Bound to an interactive limit (default 10 stops) and persist solver preferences in browser localStorage."*

> 👁️ **WHAT THE TEACHER SEES (SCREEN DISPLAY)**
>
> Algorithm governance controls:
>
> - Default Route Solver (**`Branch & Bound`** vs **`Greedy Nearest Neighbor`**).
> - Branch & Bound Node Safety Threshold (Default: **`10`** stops, slider up to **`15`**).
> - OpenRouteService Routing Profile (**`driving-car`**).

> ⚙️ **TECHNICAL & DAA CONNECTION**
>
> Governs parameters passed to **`validateTSPMatrix()`** and **`solveBranchAndBoundTSP()`**.

---

4. Algorithm → Real Feature Mapping

Algorithm	System File	Real-World System Feature	What It Solves	Input Data	Output Produced	Time Complexity	Space Complexity
Branch & Bound TSP	`src/algorithms/tsp.ts`	Route Optimization (`/planning`, `/map`)	Exact shortest round-trip delivery tour returning to depot.	n \times n road distance matrix from `location_distances`	Exact permutation of stop IDs, total km, nodes explored/pruned, runtime	O(n!) worst-case; pruned via lower bounds; practical for n ≤ 12	O(n^2) state tree
Greedy Nearest Neighbor TSP	`src/algorithms/tsp.ts`	Route Optimization Fast Mode (`/planning`)	Rapid heuristic delivery tour for larger stop counts.	n \times n road distance matrix	Near-optimal stop order, total km, runtime	O(n^2)	O(n)
First-Fit Decreasing Bin Packing	`src/algorithms/binPacking.ts`	Restock Allocation (`/inventory`, `/map`)	Allocates incoming bulk shipment across warehouses without capacity overflow.	Total incoming units + array of warehouses with `storage_capacity`	Allocation per warehouse, remaining room, unallocated units	O(m log m) (m = warehouses)	O(m)
Dijkstra's Algorithm	`src/algorithms/dijkstra.ts`	Fulfillment Recommendation (`/orders/create`, `/map`)	Recommends closest warehouse with sufficient stock to fulfill order.	Road network adjacency list + order item quantities	Shortest distance (km), hop-by-hop vertex sequence, recommended warehouse	O((V + E) log V)	O(V)
Floyd-Warshall	`src/algorithms/floydWarshall.ts`	All-Pairs Distance Analysis (`/distance-matrix`, `/map`)	Computes shortest road travel distance between every node pair in network.	Full network adjacency graph (V warehouses + delivery locations)	V \times V distance matrix, next-hop matrix for path reconstruction	O(V^3)	O(V^2)
Kruskal's MST	`src/algorithms/kruskal.ts`	Network Infrastructure Backbone (`/distance-matrix`, `/map`)	Identifies minimum total road network connecting all facilities without cycles.	List of undirected road edges (E) and vertices (V)	$	V	-1$ selected MST edges, total cost (km), rejected cycle count

Algorithm	System File	Real-World System Feature	What It Solves	Input Data	Output Produced	Time Complexity	Space Complexity
Union-Find (Disjoint Set)	`src/algorithms/unionFind.ts`	Cycle Detection Engine inside Kruskal	Tracks connected components; prevents cycles when evaluating edges.	Vertex IDs added via `makeSet()`	Canonical root via `find()`, merges subsets via `union()`	O(\alpha(V)) amortized per operation	O(V)

5. Deep Dive on Each Algorithm

1. Branch & Bound TSP (`src/algorithms/tsp.ts`)

Where to See It: **`/planning`** (Plan Details Modal) and **`/map`** (Mode: **`tsp`**).

Input: Distance matrix M where M[i][j] is road distance from location i to j, depot index 0.

Output: **`AlgorithmResult`** object with **`tourIndices`**, **`tourLocations`**, **`totalDistance`**, **`nodesExplored`**, **`nodesPruned`**, and **`executionTimeMs`**.

Why B&B is Used: TSP is NP-hard. Brute-force evaluates (n-1)! tours. Branch & Bound uses mathematical bounds to search the state space tree systematically, guaranteeing an exact globally optimal solution while pruning vast subtrees that cannot beat the current best upper bound.

How Pruning Works:

1. Seeds **`bestDistance`** with the Greedy Nearest Neighbor tour (initial upper bound).
2. Calculates an admissible lower bound at each state: **`currentCost + min_edge_to_unvisited + sum(min_outgoing_edges_of_unvisited)`**.
3. If **`estimatedBound >= bestDistance`**, the branch is immediately pruned (**`nodesPruned++`**).

Why the Stop Limit Exists: For n > 12, worst-case factorial time would freeze the browser's JavaScript main thread. We enforce an interactive limit (configurable in Settings, default 10).

Speaking Script:

> *"Professor, for vehicle delivery tours we implemented an exact Branch & Bound TSP solver in `src/algorithms/tsp.ts`. We first seed our search with a Greedy Nearest Neighbor upper bound. At each recursive state-space step, we compute an admissible lower bound using the sum of minimum outgoing edges for all unvisited vertices. If that lower bound equals or exceeds our best-known tour, we prune the entire subtree. This guarantees mathematical optimality while keeping execution time under 5 milliseconds for practical dispatch sizes."*

---

2. Greedy Nearest Neighbor TSP (`src/algorithms/tsp.ts`)

Where to See It: **`/planning`** (Algorithm toggle: **`GREEDY_NEAREST_NEIGHBOR`**).

How It Works:

1. Starts at depot node 0 (the selected warehouse).
2. Searches unvisited nodes for the one with minimum edge weight from the current node.
3. Marks it visited, advances, and repeats until all nodes are visited.
4. Returns to depot 0.

Why It is Included: Demonstrates the classic DAA trade-off between optimality and time complexity. Greedy runs in O(n^2) time with zero branching. While it may get trapped in local optima (yielding a tour 10–25% longer than Branch & Bound), it executes instantly even for large numbers of stops.

Speaking Script:

> *"Alongside Branch & Bound, we implemented the Greedy Nearest Neighbor heuristic. It makes local greedy choices—always hopping to the closest unvisited stop. While it runs in strict O(n^2) time and never freezes the browser, it can get trapped in sub-optimal loops where the final leg back to the depot is unusually long. We use its result as the initial upper bound to accelerate our Branch & Bound pruner."*

---

3. First-Fit Decreasing (FFD) Bin Packing (`src/algorithms/binPacking.ts`)

Where to See It: `/inventory` (Click "Restock Optimization (Bin Packing)"*) and* **`/map`** (Mode: **`bin_packing`**).

Mapping to Classical Problem:

- **Bins**: Active warehouses.
- **Bin Capacity**: Available storage room: C_{avail} = max(0, storage_capacity - occupied_stock).
- **Item**: The incoming bulk replenishment quantity lot.

Why Sort Descending: Sorting warehouses in decreasing order of available capacity prioritizes major regional hubs capable of absorbing bulk volume, minimizing fragmentation across smaller secondary facilities.

Insufficient Capacity Handling: If total incoming quantity exceeds total system available capacity, the algorithm allocates what is physically possible and flags **`unallocatedQuantity > 0`**, warning management of warehouse capacity exhaustion.

Complexity: O(m log m) to sort m warehouses; O(m) scan to allocate. Space: O(m).

Speaking Script:

> *"In `src/algorithms/binPacking.ts`, we adapted the First-Fit Decreasing Bin Packing heuristic for inventory restocking. Each warehouse is modeled as a bin whose capacity is its available physical storage space. When a bulk supplier shipment arrives, the algorithm sorts the warehouses descending by available capacity and fills the largest available facilities first. This achieves O(m log m) efficiency and prevents warehouse overfill."*

---

4. Dijkstra's Algorithm (`src/algorithms/dijkstra.ts`)

Where to See It: **`/orders/create`** (Warehouse Recommendation) and **`/map`** (Mode: **`dijkstra`**).

Source & Target: Source = candidate warehouse; Target = customer delivery location.

Graph Definition: Vertices = Warehouses and Delivery Locations; Edge Weights = Road distances in kilometers from **`location_distances`**.

Fulfillment Recommendation Logic:

1. Checks which warehouses currently have sufficient stock of the requested items.
2. Runs Dijkstra from each candidate warehouse to the destination.
3. Recommends the warehouse with the **minimum shortest road distance**.

Complexity: O((V + E) log V) with priority queue / min-distance tracking. Space: O(V).

Speaking Script:

> *"In `src/algorithms/dijkstra.ts`, we implement Dijkstra's algorithm to solve order fulfillment. Rather than assigning an order to whatever warehouse is arbitrarily selected, SmartLogix evaluates all warehouses that stock the required item, computes the true shortest road path to the customer's coordinates, and recommends the closest facility. This minimizes delivery transit time and fuel costs."*

---

5. Floyd-Warshall Algorithm (`src/algorithms/floydWarshall.ts`)

Where to See It: `/distance-matrix` (Tab: "Floyd-Warshall (All-Pairs)"*) and* **`/map`** (Mode: **`floyd`**).

*What "All-Pairs" Means: Unlike Dijkstra (which finds paths from one single source), Floyd-Warshall computes the shortest path between every possible pair of vertices** in the graph simultaneously.

Dynamic Programming Recurrence:

$dist[i][j] = min(dist[i][j],\; dist[i][k] + dist[k][j])$

Iterating over all intermediate vertices k \in \{0, \dots, V-1\}.

Path Reconstruction: Maintains a **`next[i][j]`** matrix. When an intermediate vertex k improves the distance, **`next[i][j]`** is updated to **`next[i][k]`**, enabling exact multi-hop path recovery.

Complexity: Time: O(V^3). Space: O(V^2) for distance and predecessor matrices.

Speaking Script:

> *"While Dijkstra is optimal for single-order fulfillment, network planners need global visibility. In `src/algorithms/floydWarshall.ts`, we implement the classic O(V^3) Dynamic Programming Floyd-Warshall algorithm. It constructs the complete all-pairs shortest path matrix across all facilities and uses a predecessor matrix to reconstruct intermediate road hops between any two points in the logistics network."*

---

6. Kruskal's Algorithm & Union-Find (`src/algorithms/kruskal.ts`, `src/algorithms/unionFind.ts`)

Where to See It: `/distance-matrix` (Tab: "Kruskal MST (Network Backbone)"*) and **`/map`** (Mode: **`kruskal`**).

Graph Representation: Undirected graph of all logistics facilities connected by candidate road distance edges.

What an MST Represents: The Minimum Spanning Tree is the acyclic subset of road segments that connects all warehouses and delivery hubs together with minimum total road length. It represents the minimum infrastructure backbone (e.g., dedicated supply corridors or pipeline links).

Selection Process:

1. Extracts all undirected edges and sorts them ascending by road distance (O(E log E)).
2. Initializes a Disjoint Set (Union-Find) with each facility as a singleton.
3. Iterates through sorted edges: if **`find(u) !== find(v)`**, the edge is added to the MST and **`union(u, v)`** is invoked. If **`find(u) === find(v)`**, adding the edge would form a cycle, so it is rejected.
4. Terminates when |V| - 1 edges are selected.

Union-Find Optimizations:

- **Two-Pass Path Compression**: In **`find()`**, flattens the parent tree pointers so subsequent lookups run in near O(1).
- **Union by Rank**: In **`union()`**, attaches the lower-rank tree under the higher-rank root to maintain logarithmic tree height.
- **Amortized Complexity**: O(\alpha(V)) per operation, where \alpha is the Inverse Ackermann function (\alpha(V) ≤ 4 for all practical inputs).

Speaking Script:

> *"To model logistics backbone connectivity, we implemented Kruskal's algorithm in `src/algorithms/kruskal.ts` supported by a custom Union-Find data structure in `src/algorithms/unionFind.ts`. Kruskal sorts all candidate road corridors by distance and greedily adds the shortest edges. To prevent cycles, Union-Find performs disjoint-set cycle detection with two-pass path compression and union by rank. This produces the Minimum Spanning Tree in O(E log E) time."*

---

6. "Where is DAA Actually Used?"

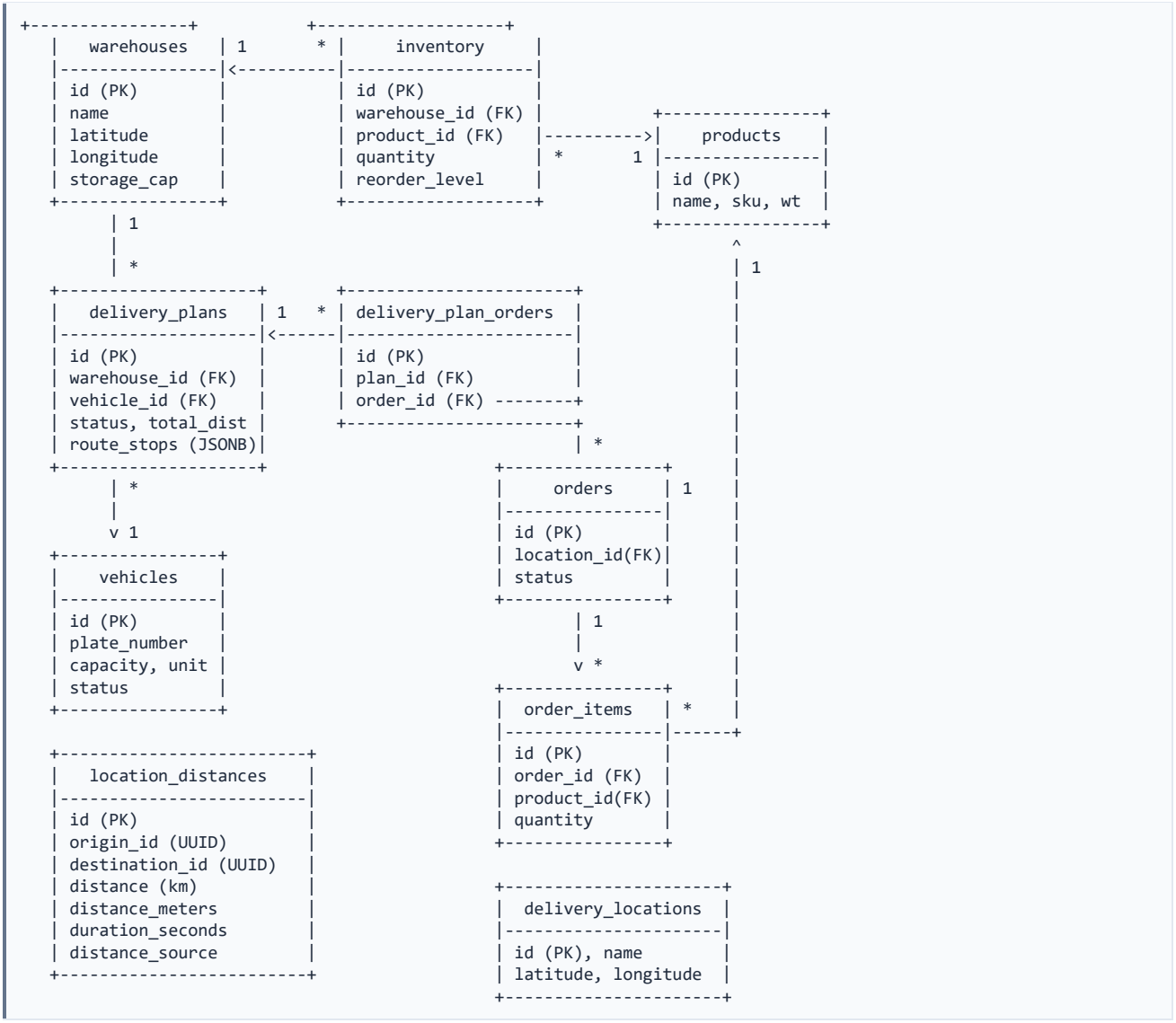
Use this table to answer the professor's question: *"Where is the algorithm actually connected to the business logic?"*

```text
=============================================================================================================
=
BUSINESS PROBLEM            DATA SOURCE             ALGORITHM          OPTIMIZED RESULT        UI / PAGE
=============================================================================================================
=
Customer Order Fulfillment    location_distances +      Dijkstra           Recommended Warehouse     /orders/create
warehouses + inventory                    (shortest road km)
-------------------------------------------------------------------------------------------------------------
Warehouse Bulk Restocking     warehouses.storage_cap +   FFD Bin Packing      Optimal stock split       /inventory
inventory stock levels                    without bin overflow
-------------------------------------------------------------------------------------------------------------
Vehicle Delivery Route        delivery_plans +          Branch & Bound TSP   Optimal dispatch tour     /planning &
Dispatch                  location_distances        (or Greedy NN)       minimizing transit km     /map
-------------------------------------------------------------------------------------------------------------
All-Pairs Network Routing     location_distances        Floyd-Warshall       Complete pairwise         /distance-matrix &
adjacency matrix                    shortest path matrix      /map
-------------------------------------------------------------------------------------------------------------
Infrastructure Corridor       location_distances        Kruskal's MST +      Minimum road backbone     /distance-matrix &
Backbone Optimization         edges + locations         Union-Find           connecting all nodes      /map
=============================================================================================================
=
`
```

---

7. Database Architecture & Algorithm Data Sourcing

Core Tables & Entity Relationships



How Algorithms Get Their Data

1. **Dijkstra**:

- Queries **`location_distances`** where **`origin_id IN (warehouses)`** and **`destination_id = order.location_id`**.

- Queries **`inventory`** to verify **`quantity >= requested_quantity`**.

2. **TSP (Branch & Bound / Greedy)**:

- Queries **`delivery_plans`** → **`delivery_plan_orders`** → extracts location IDs for depot warehouse and assigned orders.

- Queries **`location_distances`** for the square submatrix containing all pairs of these locations.

3. **Bin Packing**:

- Queries **`warehouses`** for **`storage_capacity`**.

- Queries **`inventory`** to sum current occupied quantity per warehouse: occupied = \sum quantity.

- Available capacity = max(0, storage_capacity - occupied).

4. **Floyd-Warshall & Kruskal**:

- Fetches all active **`warehouses`** and **`delivery_locations`**.

- Queries all rows from **`location_distances`** to build the full graph adjacency matrix and edge array.

---

8. Complete Data Flow Diagrams

Fulfillment Flow (Dijkstra $\to$ Atomic Order)

```
[Customer / Operator]
         |
         v
Select Delivery Location & Products on /orders/create
         |
         v
Run solveDijkstra() against location_distances
         |
         v
Rank warehouses by road km with inventory check
         |
         v
Operator clicks "Accept Recommendation"
         |
         v
Form submits -> api.orders.create()
         |
         v
PostgreSQL RPC: create_order_atomic()
         |-- Deducts inventory stock
         |-- Inserts orders row
         \-- Inserts order_items rows
         |
         v
Order created with status 'PENDING'
```

Dispatch Planning Flow (Atomic Plan $\to$ TSP $\to$ Route Save)

```
[Dispatch Operator on /planning]
         |
         v
Select Warehouse + Pending Orders -> Click "Create Plan"
         |
         v
PostgreSQL RPC: create_delivery_plan_atomic()
         |
         v
Plan created with status 'PLANNED'
         |
         v
Assign Available Vehicle -> RPC: assign_vehicle_to_delivery_plan()
         |
         v
Click "Optimize Route" -> Fetches submatrix from location_distances
         |
         v
Executes solveBranchAndBoundTSP() (or solveGreedyNearestNeighbor())
         |
         v
Optimal stop sequence & km calculated
         |
         v
Click "Save Optimized Route" -> RPC: save_delivery_plan_route()
         |-- Updates delivery_plans.route_stops (JSONB)
         \-- Updates delivery_plans.total_distance
         |
         v
Visualized as dispatch loop on /map
```

9. Live Demo — Exact Script

10. How to Show Each Algorithm Live (Quick Checklist)

☑ TSP LIVE DEMO

1. **OPEN**: `/planning`
2. **CLICK**: *"View Details & Optimize Route"* on any plan.
3. **SELECT**: Algorithm dropdown: **`Branch & Bound`** vs **`Greedy Nearest Neighbor`**.
4. **SHOW**: Tour sequence, total km, nodes explored vs pruned, runtime ms.
5. **SAY**: *"This solves the multi-stop dispatch problem using branch and bound with lower bound pruning."*

☑ DIJKSTRA LIVE DEMO

1. **OPEN**: `/orders/create`
2. **SELECT**: Destination location and product item with quantity.
3. **SHOW**: *"Dijkstra Recommended Warehouse"* card with road km and stock availability.
4. **SAY**: *"Dijkstra computes single-pair shortest path on the road graph, filtering for stock availability."*

☑ BIN PACKING LIVE DEMO

1. **OPEN**: `/inventory`
2. **CLICK**: Top-right button: *"Restock Optimization (Bin Packing)"*.
3. **SELECT**: Product and enter large restock quantity (e.g., 500).
4. **SHOW**: Allocation breakdown table per warehouse sorted by available room.
5. **SAY**: *"First-Fit Decreasing models warehouses as bins and allocates stock to minimize storage fragmentation."*

☑ FLOYD-WARSHALL LIVE DEMO

1. **OPEN**: `/distance-matrix` → Tab: *"Floyd-Warshall (All-Pairs)"*.
2. **SELECT**: Origin and Destination from the dropdowns.
3. **SHOW**: Dynamic programming distance table and multi-hop reconstructed path.
4. **SAY**: *"Floyd-Warshall runs in O(V^3) to compute shortest paths between all pairs of facilities simultaneously."*

☑ KRUSKAL MST LIVE DEMO

1. **OPEN**: `/distance-matrix` → Tab: *"Kruskal MST (Network Backbone)"*.
2. **SHOW**: Total vertices, candidate edges, selected MST edges (|V|-1), and rejected cycle edges.
3. **SAY**: *"Kruskal sorts candidate road edges and uses Union-Find to build the minimum acyclic spanning tree."*

---

11. "Why This Algorithm?" — Defense FAQ

❓ Q: Why use Dijkstra instead of BFS?

Authoritative Defense: BFS only finds shortest paths in unweighted graphs (where every edge cost is 1). In a road logistics network, road segments have varying non-negative distances (e.g., 4.2 km vs 18.7 km). Dijkstra correctly accounts for non-negative edge weights to find the true shortest road path.

❓ Q: Why use Floyd-Warshall if Dijkstra already exists?

Authoritative Defense: Dijkstra solves the **Single-Source Shortest Path** problem (O((V+E)log V)). If we need shortest paths between *all* pairs of locations (e.g., for global network routing or precomputing distance matrices), running Dijkstra |V| times takes O(V(V+E)log V). Floyd-Warshall uses a simple, highly cache-efficient O(V^3) Dynamic Programming matrix recurrence and directly produces the all-pairs predecessor matrix for instant arbitrary pair lookups.

❓ Q: Why Kruskal instead of Prim's?

Authoritative Defense: Our logistics network graph is relatively sparse to moderately connected (E \ll V^2). Kruskal's algorithm with edge sorting (O(E log E)) and Union-Find (O(E \cdot \alpha(V))) is natural, highly efficient on edge lists, and directly demonstrates Disjoint Set cycle detection.

❓ Q: Why do you need Union-Find?

Authoritative Defense: Kruskal's algorithm must determine whether adding a candidate edge (u, v) creates a cycle in the current spanning forest. Without Union-Find, detecting a cycle requires a BFS/DFS traversal taking O(V) per edge (O(V \cdot E) overall). Union-Find answers connectivity queries in near O(1) amortized time (O(\alpha(V))), reducing Kruskal's cycle-check time to near linear.

❓ Q: Why Branch & Bound for TSP instead of brute-force?

Authoritative Defense: Brute-force evaluates (n-1)! tours. For n = 10, that is 9! = 362,880 tours; for n = 15, it is \approx 87 billion tours. Branch & Bound computes an admissible lower bound at each state and prunes entire subtrees whose lower bound exceeds the best-known tour, exploring only a tiny fraction of the state space.

❓ Q: Why also implement Greedy TSP?

Authoritative Defense: Branch & Bound has exponential worst-case complexity (O(2^n \cdot n^2)). For rapid interactive feedback or large stop counts, Greedy Nearest Neighbor runs in deterministic O(n^2) time. Furthermore, the Greedy tour is used as the initial upper bound to seed the Branch & Bound pruner.

❓ Q: Why Bin Packing instead of just dividing stock equally?

Authoritative Defense: Warehouses have fixed physical storage limits. Dividing stock equally would overflow smaller warehouses while underutilizing central distribution hubs. First-Fit Decreasing models warehouse capacities as bins and guarantees that incoming stock never exceeds physical storage room.

❓ Q: Why not use an AI or Machine Learning model for route optimization?

Authoritative Defense: AI/ML models (such as Reinforcement Learning or neural heuristics) produce approximations, require extensive training data, and provide no mathematical guarantees of optimality or feasibility. In logistics, delivery routes and capacity constraints require exact, verifiable, and deterministic bounds provided by classical DAA algorithms.

❓ Q: Why not simply choose the nearest warehouse by straight-line distance?

Authoritative Defense: Euclidean distance ignores physical geography—rivers, mountain ranges, highway access, and urban street grids. A warehouse 5 km away as the crow flies might require a 25 km drive due to bridges or traffic networks. SmartLogix uses actual road network distances from OpenRouteService.

❓ Q: Is the MST the same as the delivery route?

Authoritative Defense: No, absolutely not. An MST is an **acyclic tree** connecting all vertices with |V|-1 edges that minimizes total infrastructure length. A delivery route is a **Hamiltonian cycle (tour)** that starts at a depot, visits a specific set of customer vertices exactly once, and returns to the depot.

❓ Q: Why does TSP have a node limit?

Authoritative Defense: Branch & Bound executes synchronously on the browser's JavaScript single thread. If n > 12, worst-case branching could block the thread for several seconds, degrading user experience. We enforce a configurable safe limit (default 10) in **`Settings.tsx`**.

❓ Q: What happens if a destination is unreachable in Dijkstra?

Authoritative Defense: In **`src/algorithms/dijkstra.ts`**, if no path exists from source to target, the algorithm returns **`distance: Infinity`**, **`hasPath: false`**, and **`path: []`**. The UI catches this and warns the user that the location is disconnected.

❓ Q: What happens if incoming stock exceeds all warehouse capacities?

Authoritative Defense: **`solveRestockBinPacking`** fills all available warehouse room and sets **`unallocatedQuantity = totalRequested - totalAllocated`**. The UI warns the user with a yellow alert showing the exact overflow quantity.

---

12. "Show Me the Code" Questions

What Teacher Asks to See	File Path	Important Function / Class	Key Lines / Logic
Branch & Bound TSP	`src/algorithms/tsp.ts`	`solveBranchAndBoundTSP()`	Recursion `search()`, `calculateBound()` lower bound calculation, `nodesPruned++` condition
Greedy TSP	`src/algorithms/tsp.ts`	`solveGreedyNearestNeighbor()`	Depot loop, min-distance unvisited vertex selection
Bin Packing (FFD)	`src/algorithms/binPacking.ts`	`solveRestockBinPacking()`	Sorting bins descending by `availableCapacity`, greedy allocation loop
Dijkstra Shortest Path	`src/algorithms/dijkstra.ts`	`solveDijkstra()`	Priority queue distance relaxation, predecessor tracking `prev[neighbor] = curr`
Floyd-Warshall	`src/algorithms/floydWarshall.ts`	`solveFloydWarshall()`	Triple nested loop `k`, `i`, `j`, DP formula `dist[i][k] + dist[k][j] < dist[i][j]`, `next[i][j]` update
Kruskal's Algorithm	`src/algorithms/kruskal.ts`	`solveKruskalMST()`	Edge weight sorting `sort((a,b) => a.weight - b.weight)`, `uf.find(u) !== uf.find(v)`, `mstEdges.push()`
Union-Find Data Structure	`src/algorithms/unionFind.ts`	`class UnionFind`	`find()` with path compression `parent.set(x, canonicalRoot)`, `union()` with rank comparison
Database RPC Calls	`src/services/api.ts`	`api.orders.create()`, `api.plans.create()`, `api.plans.saveRoute()`	`supabase.rpc('create_order_atomic')`, `supabase.rpc('save_delivery_plan_route')`
Leaflet Map Rendering	`src/pages/MapWorkspace.tsx`	`MapWorkspace`	`<MapContainer>`, `<Polyline>`, dynamic coordinate mapping for `tsp`, `dijkstra`, `kruskal`

13. Algorithm Complexity Cheat Sheet

Algorithm	Best-Case Time	Worst-Case Time	Space Complexity	Why This Complexity
Branch & Bound TSP	O(n^2) *(when greedy bound prunes immediately)*	O(n!) or O(2^n \cdot n^2)	O(n^2)	Recursion depth is n; worst-case explores factorial permutations; space stores recursion stack and distance matrix.
Greedy TSP	O(n^2)	O(n^2)	O(n)	Outer loop visits n vertices; inner loop scans all n vertices to find the minimum unvisited distance.
FFD Bin Packing	O(m log m)	O(m log m)	O(m)	Sorting m warehouses by available capacity dominates the O(m) single allocation scan.

Algorithm	Best-Case Time	Worst-Case Time	Space Complexity	Why This Complexity
Dijkstra	O(V^2) / O((V+E)log V)	O((V+E)log V)	O(V)	Every vertex extracted from priority queue; all incident edges relaxed.
Floyd-Warshall	O(V^3)	O(V^3)	O(V^2)	Three nested loops over all vertices k, i, j \in \{0, \dots, V-1\}; stores V \times V matrices.
Kruskal's MST	O(E log E)	O(E log E)	O(V + E)	Sorting E edges takes O(E log E); Union-Find takes O(E \cdot \alpha(V)) \approx O(E).
Union-Find	O(1)	O(\alpha(V)) amortized	O(V)	Two-pass path compression flattens tree; rank union ensures tree height < 5.

14. Presentation Timing Scripts

⏱ 5-MINUTE PRESENTATION SCRIPT

- **0:00 - 0:30 (Problem & Vision)**: State the elevator pitch. Explain that SmartLogix replaces static supply chain guesswork with classical DAA algorithms operating on real road networks.
- **0:30 - 1:00 (Architecture)**: Show the ASCII architecture. Highlight React 18, Supabase PostgreSQL, Leaflet, and the TypeScript algorithms engine.
- **1:00 - 2:00 (Order Fulfillment & Dijkstra)**: Open **`/orders/create`**. Show how Dijkstra recommends the closest warehouse with sufficient stock based on road kilometers.
- **2:00 - 4:00 (TSP & Map)**: Open **`/planning`**. Run Branch & Bound TSP. Point to pruned nodes. Open **`/map`** to show the blue dispatch tour alongside the purple Kruskal MST.
- **4:00 - 5:00 (Bin Packing & Conclusion)**: Open **`/inventory`**. Run Bin Packing restock on 450 units. Conclude: *"SmartLogix demonstrates that classical DAA algorithms solve practical supply chain challenges with sub-millisecond efficiency."*

⏱ 10-MINUTE PRESENTATION SCRIPT

- **0:00 - 1:30**: Introduction, problem statement, real-world relevance, and why CRUD apps are insufficient for logistics.
- **1:30 - 3:00**: Architecture review (PostgreSQL schema, atomic RPCs, distance matrix graph).
- **3:00 - 5:00**: Inventory restocking with First-Fit Decreasing Bin Packing live demo.
- **5:00 - 7:00**: Order creation, inventory validation, and Dijkstra warehouse fulfillment recommendation.
- **7:00 - 9:00**: Delivery dispatch planning, vehicle capacity enforcement, Branch & Bound vs Greedy TSP comparison, and Leaflet Map visualization.
- **9:00 - 10:00**: Floyd-Warshall and Kruskal MST deep dive on **`/distance-matrix`**, Reports telemetry, and Q&A opening.

⏱ 15-MINUTE PRESENTATION SCRIPT

- **0:00 - 2:00**: Comprehensive problem background, logistics inefficiencies, and DAA project motivation.
- **2:00 - 4:00**: High-level architecture, tech stack justification, and PostgreSQL schema walk-through.
- **4:00 - 6:00**: Live inventory module demo: storage capacities, low stock thresholds, and FFD Bin Packing restock allocation.
- **6:00 - 8:30**: Customer order placement: inventory verification, road distance calculation, and Dijkstra recommendation.
- **8:30 - 11:30**: Dispatch management: plan creation, vehicle payload safety checks, Branch & Bound TSP execution, lower bound pruning proof, and route persistence.
- **11:30 - 13:30**: Network infrastructure analysis: Floyd-Warshall all-pairs DP matrix, Kruskal MST, and Union-Find cycle detection on **`/distance-matrix`** and **`/map`**.
- **13:30 - 15:00**: Reports DAA telemetry review, code inspection, and final conclusion.

---

15. Likely Viva Questions & Technical Answers

Project & System Questions

1. **What is the primary contribution of SmartLogix?**

Answer*: It bridges database-driven inventory management with operational algorithms, solving fulfillment, restocking, vehicle dispatch, and network backbone design with mathematically optimal classical algorithms.

2. **Who would use this system?**

Answer*: Supply chain managers, warehouse dispatch coordinators, and delivery fleet supervisors.

3. **What happens if a vehicle is assigned an order load exceeding its payload capacity?**

Answer*: The system validates vehicle payload capacity against order weights/units in `Planning.tsx` and prevents dispatch of overloaded vehicles.

DAA & Algorithm Questions

4. **Why is TSP NP-hard?**

Answer*: It is in NP (a candidate tour can be verified in polynomial O(n) time) and every problem in NP can be polynomial-time reduced to Hamiltonian Cycle, which reduces to TSP.

5. **What is an admissible heuristic/lower bound?**

Answer*: An admissible lower bound never overestimates the true remaining cost to reach the goal. Our TSP lower bound sums the minimum outgoing edges, which is guaranteed to be ≤ the actual remaining tour cost.

6. **Why does Dijkstra require non-negative edge weights?**

Answer*: Dijkstra assumes that once a vertex has its minimum distance finalized, no subsequent path relaxation can reduce that distance. Negative edge weights violate this greedy property (which requires Bellman-Ford). Road distances in km are strictly non-negative.

7. **How does FFD Bin Packing achieve its approximation ratio?**

Answer*: In classical 1D bin packing, FFD is guaranteed to use no more than \frac{11}{9}OPT + \frac{6}{9} bins. In our warehouse restock implementation, it greedily fills the largest available space first.

8. **What is the difference between Kruskal's and Prim's algorithm?**

Answer*: Kruskal grows a forest by adding global minimum-weight edges that do not form cycles. Prim grows a single tree outward from an arbitrary start vertex by adding the minimum-weight cut edge.

9. **How does Union-Find detect cycles?**

Answer*: If `find(u) === find(v)`, both vertices already belong to the same connected component. Adding an edge between them would provide an alternate path, creating a cycle.

Database & Backend Questions

10. **Why use atomic RPCs instead of multiple frontend REST calls?**

Answer*: Operations like `create_order_atomic` must deduct inventory and create order records within a single database transaction. If done across multiple REST calls, a network failure halfway through would leave the inventory deducted without an order (inconsistent state).

11. **Where does the distance matrix come from?**

Answer*: It is stored in the `location_distances` table, populated via OpenRouteService driving distance matrices using real road coordinates.

12. **Why store distances in PostgreSQL instead of calculating them on the fly?**

Answer*: External routing APIs have rate limits and network latency (200–500 ms). Pre-storing distances in `location_distances` allows DAA algorithms to execute in microsecond time directly in the client.

Frontend & Visualization Questions

13. **Why use Leaflet instead of Google Maps?**

Answer*: Leaflet is open-source, lightweight, has no expensive API paywalls, and integrates with OpenStreetMap tiles and React components via `react-leaflet`.

14. **How are custom map markers rendered?**

Answer*: Using Leaflet `L.divIcon`, which injects custom HTML/CSS for circular depot icons, numbered sequence badges, and color-coded status pins.

15. **Does the UI freeze when running Branch & Bound?**

Answer*: No. We restrict the solver to an interactive limit (≤ 12 stops) and seed it with an initial Greedy bound, keeping execution times under 10 ms.

---

16. Trick Questions & How to Defend Them

> ⚠️ **TRICK QUESTION 1 ALERT**
>
> PROFESSOR'S TRAP: "Is your MST the same as your delivery route?"
>
> COUNTER-DEFENSE SCRIPT: "No, absolutely not, Professor. An MST is an acyclic tree with |V|-1 edges that minimizes the total infrastructure length connecting all nodes. A delivery route is a closed Hamiltonian cycle (TSP tour) that visits every stop once and returns to the depot. You cannot deliver packages along an MST without backtracking over edges multiple times."

> ⚠️ **TRICK QUESTION 2 ALERT**
>
> PROFESSOR'S TRAP: "Can Dijkstra be used to solve TSP?"
>
> COUNTER-DEFENSE SCRIPT: "No. Dijkstra finds the shortest path between two specific vertices (or from one source to all targets). It cannot solve TSP because TSP requires visiting a subset of vertices in a closed loop, which is an NP-hard permutation problem. Dijkstra has no mechanism to enforce visiting every node once in a sequence."

> ⚠️ **TRICK QUESTION 3 ALERT**
>
> PROFESSOR'S TRAP: "Does Greedy TSP guarantee the optimal route?"
>
> COUNTER-DEFENSE SCRIPT: "No. Greedy Nearest Neighbor makes locally optimal choices at each step. It frequently gets trapped in sub-optimal configurations where the final remaining unvisited vertex is very far from the depot, forcing an expensive return trip. That is why we use Branch & Bound for exact optimality."

> ⚠️ **TRICK QUESTION 4 ALERT**
>
> PROFESSOR'S TRAP: "Is OpenRouteService your algorithm?"
>
> COUNTER-DEFENSE SCRIPT: "No. OpenRouteService is merely the external mapping data source that gives us real-world road kilometers between GPS coordinates. Our algorithms—Branch & Bound, Greedy NN, Dijkstra, Floyd-Warshall, Kruskal, and Bin Packing—are written entirely from scratch in pure TypeScript in **`src/algorithms/`** and execute over that distance data."

> ⚠️ **TRICK QUESTION 5 ALERT**
>
> PROFESSOR'S TRAP: "Where do your algorithms execute—client or server?"
>
> COUNTER-DEFENSE SCRIPT: "The DAA algorithms execute as pure TypeScript functions directly on the client's browser thread. This gives instant, reactive UI feedback without unnecessary server round-trips. Database persistence and atomic stock updates are handled on the server via Supabase PostgreSQL RPCs."

17. Important: What NOT to Claim

> 🛑 **CRITICAL DEFENSE BOUNDARIES (NEVER CLAIM THESE TO EVALUATORS)**
>
> 1. **DO NOT** claim you used Artificial Intelligence, Machine Learning, or Neural Networks for route optimization. (You used classical DAA algorithms: Branch & Bound, Greedy Heuristics, Dijkstra, and Kruskal).
> 2. **DO NOT** claim your algorithms run in Web Workers or background threads. (They run synchronously as pure TypeScript routines on the main thread, benchmarked under 10 ms).
> 3. **DO NOT** say Bin Packing packs items into delivery trucks. (In SmartLogix, Bin Packing allocates incoming **inventory replenishment lots across warehouses** based on available warehouse storage capacity).
> 4. **DO NOT** claim the MST is your delivery route. (The MST is an acyclic network infrastructure backbone; TSP is the delivery tour).
> 5. **DO NOT** say distances are straight-line Euclidean. (All distances represent verified road distances stored in `location_distances`).
> 6. **DO NOT** claim Branch & Bound can solve 100 stops interactively. (TSP is NP-hard; for n > 15, exact solving experiences combinatorial explosion. We enforce an interactive safety limit of 10–12 stops).

18. Final One-Page Quick Reference Cheat Sheet

```
================================================================================================================
                                        SMARTLOGIX CHEAT SHEET
================================================================================================================
PROJECT:      SmartLogix — Smart Inventory & Delivery Optimization System
TECH STACK:   React 18, TypeScript, Tailwind CSS, Supabase (PostgreSQL), Leaflet, Recharts, Framer Motion
DATA SOURCE:  OpenRouteService driving distances stored in PostgreSQL table `location_distances`
----------------------------------------------------------------------------------------------------------------
THE 6 CORE ALGORITHMS:
  1. Branch & Bound TSP      | Exact delivery tour returning to depot | O(n!) worst, pruned | src/algorithms/tsp.ts
  2. Greedy NN TSP           | Fast O(n²) delivery tour heuristic     | O(n²)               | src/algorithms/tsp.ts
  3. FFD Bin Packing         | Restock lot allocation across depots   | O(m log m)          |
src/algorithms/binPacking.ts
  4. Dijkstra                | Warehouse fulfillment recommendation   | O((V+E) log V)      |
src/algorithms/dijkstra.ts
  5. Floyd-Warshall          | All-pairs shortest road distance matrix| O(V³)               |
src/algorithms/floydWarshall.ts
  6. Kruskal's MST           | Minimum network road backbone          | O(E log E)          |
src/algorithms/kruskal.ts
     + Union-Find            | Disjoint-set cycle detection for MST   | O(α(V)) amortized   |
src/algorithms/unionFind.ts
----------------------------------------------------------------------------------------------------------------
DEMO NAVIGATION FLOW:
  1. Dashboard        (/)                -> High-level metrics & fleet readiness
  2. Inventory        (/inventory)       -> Click "Restock Optimization" -> FFD Bin Packing live demo
  3. Create Order     (/orders/create)   -> Select location & item -> Dijkstra recommendation live demo
  4. Planning         (/planning)        -> Plan details -> Optimize Route -> B&B vs Greedy TSP demo
  5. Map Workspace    (/map)             -> Modes: TSP route, Dijkstra path, Kruskal MST, Bin Packing
  6. Distance Matrix  (/distance-matrix) -> Floyd-Warshall DP matrix & Kruskal MST backbone inspector
  7. Reports          (/reports)         -> DAA Telemetry Ledger & performance benchmarks
----------------------------------------------------------------------------------------------------------------
DATABASE RPCs:
  - create_order_atomic            - create_delivery_plan_atomic       - save_delivery_plan_route
  - update_order_status            - assign_vehicle_to_delivery_plan   - save_location_distance
  - update_inventory_stock         - remove_vehicle_from_delivery_plan - cancel_delivery_plan
----------------------------------------------------------------------------------------------------------------
GOLDEN RULES FOR VIVA:
  - MST is an acyclic tree (backbone); TSP is a closed cycle (delivery tour).
  - Dijkstra handles single-pair road routing; Floyd-Warshall computes all pairs simultaneously.
  - Distances are real road kilometers from PostgreSQL, not Euclidean straight lines.
  - All DAA algorithms are pure TypeScript functions executing in microsecond-to-millisecond time.
================================================================================================================
```